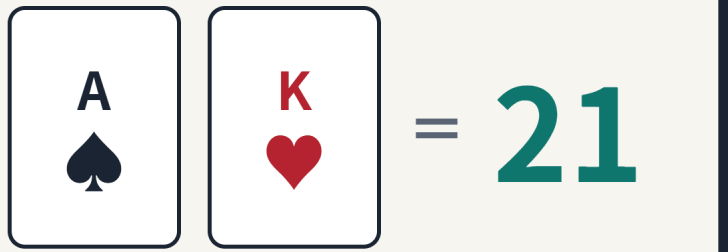
エンジニア研修 ・ オブジェクト指向 開発演習

ブラックジャック演習

まとめて書いてから、
責務を見つけて分ける



責務の分割　カプセル化（不変条件）　多態性

© 2026 by yo1000 | Yoichi KIKUCHI is licensed under CC BY-ND 4.0

01

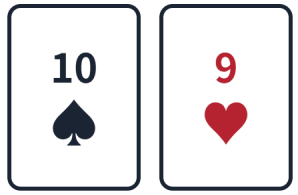
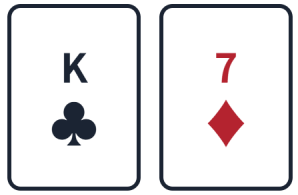
ルール

今回のブラックジャックを知る

ルール

21 に近いほうが勝ち

あなた（プレイヤー）とディーラーの1対1で勝負します。手札の点数の合計が、**21 を超えずに 21 に近いほう**が勝ちです。

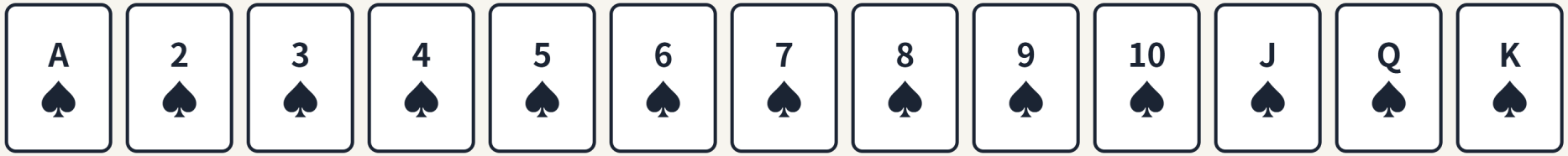
あなた		**19 点** 勝ち
ディーラー		**17 点** 負け

ただし、21 を超えたら（バースト）、その時点で負けです。

ルール

カードの点数

トランプから**ジョーカーを除いた52枚**を使います。スート（マーク）は、点数に関係しません。



カード	点数
2 〜 10	数字のとおり
J・Q・K（絵札）	10 点
A	1 点か 11 点の、都合のよいほう

ルール

ゲームの流れ

1

配る

あなたとディーラーに2枚ずつ配ります。ディーラーの2枚目は伏せておきます。

2

あなたの番

もう1枚引く（Hit）か、やめる（Stand）かを選びます。何度でも引けます。

3

ディーラーの番

伏せたカードを開き、決まりに従ってカードを引きます。

4

勝敗を決める

点数を比べて、勝ち・負け・引き分けを決めます。

21 を超えたら（バースト）、その時点で負けです。

バーストせずに同じ点数なら、引き分け（Push）です。

ルール

1人で遊ぶ

画面にあなたの手札と点数が表示されます。もう1枚引くなら **h**（Hit）、やめるなら **s**（Stand）を入力します。

[h]it

もう1枚引く

[s]tand

ここでやめる

```
You: K of Spades, 5 of Clubs (15)
[h]it or [s]tand? > h
```

ルール

ディーラーは、引くかどうかを選ばない

ディーラーは、点数が 17 点以上になるまで必ず引き、17 点以上になったら必ず止まります。



ディーラーの点数	すること
16 点以下	必ずもう1枚引く
17 点以上	必ず止まる

ルール

勝ち負けの決め方

あなたの番が、ディーラーの番より先に終わります。勝ち負けは、次の順に確かめて決めます。

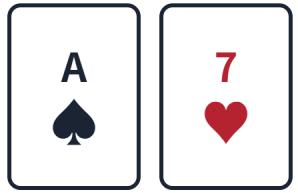
1. あなたがバーストした → その時点で**あなたの負け**。ディーラーはカードを引かない
2. ディーラーがバーストした → **あなたの勝ち**
3. どちらもバーストしていない → 21 に近いほうの勝ち。同じ点数なら**引き分け**（Push）

あなたがバーストした時点で勝負は決まるので、「両方がバースト」にはならない。

ルール ・ 遊び方の例

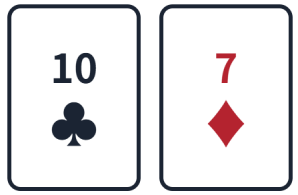
例1：A を 11 点として数える

あなた



18 点 Stand

ディーラー



17 点 止まる

結果：A を 11 点と数えて 18 点。17 点のディーラーに勝ち

ルール ・ 遊び方の例

例2：A を 1 点として数える



結果：A を 11 点のままだと 25 点になるので、A を 1 点にして 15 点。続けて引いて 20 点

ルール ・ 遊び方の例

例3：バーストする



結果：21 を超えたので、ここで負け。ディーラーはカードを引かず、あとから結果が変わることもない

02

進め方

まとめて書いてから、分ける

進め方

今回は、分け方を自分で決める

これまでの演習では、作るクラスが先に決まっていました。今回は、まず動くものを作り、その流れを見ながら分け方を考えます。

これまで

部品から作る

作るクラスと役割が、先に決まっている

作っている間は、全体の流れやクラス同士のつながりが見えにくい

今回

流れから分ける

まず、すべての処理を main に書いて動かす

動く流れを見ながら、どこで分けるかを自分たちで決める

先に全体を動かすと、何と何がつながっているかが見える。

進め方

4つの段階で分ける

一度に分けようとせず、小さな段階を順に進みます。どの段階でも、ゲームの動きは変えません。

1

main に書き下す

クラスもメソッドも作らず、すべてを main に書いて動かす

2

メソッドに切り出す

ひとまとまりの処理に、名前をつけて切り出す

3

クラスに切り出す

いつも一緒に使うデータと処理を、1つにまとめる

4

役割で分ける

種類ごとに違うところを、それぞれに任せる

名前をつける → まとめる → 任せる、の順に、少しずつ形を変えていく。

進め方

切り出す手がかり

コードの中に次のようなところを見つけたら、切り出すきっかけになります。

見つけたもの	切り出し方
同じ処理が、2か所以上に書かれている	メソッドにして、1か所にまとめる
コメントで「〜する」と説明しているかたまり	メソッドにする（コメントが名前の候補）
いつも一緒に受け渡ししているデータ	クラスにまとめる
同じデータを読み書きするメソッドの集まり	データと一緒に、クラスに移す
種類によって、if で書き分けている処理	種類ごとのクラスに任せる（多態性）

進め方

動きを変えずに分ける

切り出すたびに、**同じシードと同じ入力**で動かし、表示が変わらないことを確かめます。切り出す前の表示を before.txt に保存しておきます。

```
./mvnw -q compile
printf 'h\ns\n' | java -cp target/classes blackjack.BlackjackApp 194 > before.txt

# （切り出したあと）
./mvnw -q compile
printf 'h\ns\n' | java -cp target/classes blackjack.BlackjackApp 194 > after.txt
diff before.txt after.txt
```

ポイント	理由
引数でシード（194）を渡す	山札の切り方が、毎回同じになる
printf で入力を渡す	Hit・Stand の操作が、毎回同じになる
diff で何も表示されない	切り出しの前後で、動きが変わっていない

※ シード（seed）：乱数（ランダムな値）を作るときの「元になる数」です。同じシードを渡すと、毎回同じ順番で同じ乱数が出てきます。

03

課題

書いて、分けて、確かめる

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 課題1

ルールの中にある法則を見つけよう

3〜4人で以下を話し合いましょう。

1. 手札の点数は、どう数えればよいか。A が2枚以上あるときはどうなるか

2. プレイヤーとディーラーで、同じところと違うところは何か

3. 「勝ち・負け・引き分け」は、何をどの順に確かめれば決まるか

4. ルールに出てくる「もの・名詞」は何か。それぞれが「すること・動詞」は何か
   （それぞれ、クラス・フィールド・メソッドの候補になることをイメージしながら考えてください）

ヒント：4は、ルールの説明に出てくる名詞と動詞を拾い出すところから始めます。カード、山札、手札……と、思いつくだけ挙げてみましょう。

Maven Wrapper を取得する

Maven Wrapper を直接ダウンロードし、使う Maven のバージョンを指定します。

```
WRAPPER_VERSION=3.3.4
MAVEN_VERSION=3.10.0

mkdir blackjack && cd blackjack

curl -fsSLO "https://repo.maven.apache.org/maven2/org/apache/maven/wrapper/maven-wrapper-
distribution/${WRAPPER_VERSION}/maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip"
unzip -l maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip
unzip maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip
rm maven-wrapper-distribution-${WRAPPER_VERSION}-only-script.zip

mkdir -p .mvn/wrapper
cat > .mvn/wrapper/maven-wrapper.properties <<EOF
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/${MAVEN_VERSION}/apache-
maven-${MAVEN_VERSION}-bin.zip
EOF

chmod +x mvnw
```

準備 ・ プロジェクトを用意する（2/3）

Maven Wrapper でひな形を作る

取得した mvnw で、Maven 公式のテンプレート（archetype）からプロジェクトを生成します。blackjack フォルダの中で実行してください。

```
./mvnw archetype:generate \
  -DarchetypeGroupId=org.apache.maven.archetypes \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DarchetypeVersion=1.5 \
  -DjavaCompilerVersion=25 \
  -DgroupId=com.example \
  -DartifactId=blackjack \
  -Dpackage=blackjack \
  -DinteractiveMode=false

mv blackjack/* ./ && rm -rf blackjack/
```

archetype:generate は、artifactId と同じ名前（blackjack）のフォルダを作って、その中にプロジェクトを生成します。最後の行で、生成された中身を blackjack の直下に移しています。

-DjavaCompilerVersion=25 で、使う Java のバージョンを 25 に指定しています（指定しないと 17 になります）。あらかじめ JDK 25 をインストールしておいてください。

準備 ・ プロジェクトを用意する（3/3）

.gitignore を作る

ビルド結果や IDE の設定ファイルを、Git の管理から外します。blackjack フォルダの中で実行してください。

```
cat > .gitignore <<'EOF'
target/
.idea/
*.iml
.vscode/
.DS_Store
mvnwDebug*
before.txt
after.txt
EOF
```

できあがる構成

```
blackjack/
├── .gitignore
├── .mvn/wrapper/
│   └── maven-wrapper.properties
├── mvnw
├── mvnw.cmd
├── pom.xml
└── src/
    ├── main/java/blackjack/
    └── test/java/blackjack/
```

.mvn フォルダは Git の管理から外さないでください。使う Maven のバージョンが書かれています。

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 課題2

main に書き下す

BlackjackApp クラスの main メソッドの中に、ゲームのすべてを書いて動かします。まずは動けばよく、きれいに書く必要はありません。

1. 新しいクラスもメソッドも作らず、main 1つの中に書く
2. カードは int（0〜51 の番号）や String で表してよい
3. 実行時の引数でシードを受け取り、new Random(シード) と Collections.shuffle で山札を切る
4. 画面の表示は、次のページの表示例に合わせる

完成したら、課題3以降で比べるために before.txt を作っておく（進め方「動きを変えずに分ける」のコマンド）

演習 ・ 課題2

画面の表示例

画面の表示はすべて英語にします。カードは「7 of Hearts」のように、ランクとスートで表示します。

```
$ ./mvnw -q compile
$ java -cp target/classes blackjack.BlackjackApp 194
Dealer: 10 of Spades, [hidden]
You: J of Hearts, 8 of Hearts (18)
[h]it or [s]tand? > h
You drew A of Spades.
You: J of Hearts, 8 of Hearts, A of Spades (19)
[h]it or [s]tand? > s
Dealer: 10 of Spades, 4 of Hearts (14)
Dealer drew 4 of Diamonds.
Dealer: 10 of Spades, 4 of Hearts, 4 of Diamonds (18)

You: 19 / Dealer: 18
You win!
```

バーストしたら「Bust!」と表示します。最後に両者の点数と、You win! ・ You lose... ・ Push. のどれかを表示します。

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 課題3

メソッドに切り出す

課題2の main から、ひとまとまりの処理を static メソッドとして切り出します。

1. main を読み、ひとまとまりの処理に「〜する」と名前をつける
2. 同じ処理が2か所にあれば、1つのメソッドにまとめる
3. 1つ切り出すごとにコンパイルして動かし、diff before.txt after.txt で表示が変わらないことを確かめる

条件：main を読むだけで、ゲームの流れ（配る → あなたの番 → ディーラーの番 → 勝敗）がわかるようにする

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 課題4

クラスに切り出す

課題3で、いつも同じデータを受け取っていたメソッドを、そのデータと一緒にクラスへまとめます。int のカードも、ランクとスートを持つものに置き換えます。

1. フィールドはすべて private にし、手札や山札の List を外から直接変更できないようにする
2. カードは、ランクとスートが必ずそろっている
3. 山札から同じカードは2度出ず、空の山札からは引けない
4. 手札の点数は、いつもルールどおりに数えられる

ヒント：山札は課題3と同じ順（スートごとに A, 2, …, K）に並べてから切ると、同じシードで同じ表示になる

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 課題5

役割で分ける

playerTurn と dealerTurn は、よく似ていました。違うところだけをそれぞれに任せ、勝敗とゲームの進行もクラスにします。

1. playerTurn と dealerTurn を並べ、同じところと違うところに印をつける
2. 違うところ（もう1枚引くか）だけを、あなたとディーラーがそれぞれ決められるようにする
3. 勝ち・負け・引き分けを表すものを作り、勝敗の決め方をそこにまとめる
4. ゲームを進めるクラスを作り、main は部品を組み立てて始めるだけにする

条件：ゲームを進めるクラスは、相手があなたかディーラーかを if で書き分けない

課題

実装で意識すること

プラクティス	このゲームでの例
責務の分割 1つのクラスは1つのことを知る	点数の数え方は Hand、引くかの決め方は Player と Dealer 勝敗の決め方は Result、進める順番は BlackjackGame が知っている
カプセル化 不変条件を守る	カードはランクとスートが必ずそろい、山札から同じカードは2度出ない 手札の点数は持たずに、いつもカードから数え直す
多態性 種類を知らずに任せる	BlackjackGame は、あなたとディーラーを区別せずに playTurn() を呼ぶ

データとそれを扱う処理は一緒に置き、種類ごとの違いはそれぞれに任せる。

課題 ・ ふりかえり

最初の main と比べてみよう

課題2と課題5のコードを並べて、次のことを話し合いましょう。

1. 「ディーラーは 18 点以上になるまで引く」に変えるとき、それぞれどこを直すか
2. 点数の数え方が正しいかを確かめたいとき、それぞれどうやって確かめるか
3. 課題4・5で分けた境目は、課題1で見つけた「もの」と同じだったか

分けた境目が、変更とテストのしやすさにつながっている。

課題

課題の進め方

まずルールを観察して法則を見つけます。そのあと、動くものを1つの main に書き、少しずつ分けていきます。

課題1

法則を話し合う

3〜4人のグループで

課題2

main に書き下す

BlackjackApp の main だけで

課題3

メソッドに切り出す

かたまりに名前をつける

課題4

クラスに切り出す

データと処理をまとめる

課題5

役割で分ける

違いをそれぞれに任せる

発展・付録

仕様を変える・テストする

早く終わった人向け

演習 ・ 発展課題

仕様を変えてみる

早く終わった人は、次の変更に挑戦しましょう。変更したら、直したクラスとメソッドの数を数えてください。

1. 最初の2枚で 21 点（ブラックジャック）なら、ほかの 21 点より強いことにする
2. 決まった作戦で自動で遊ぶプレイヤーを作り、100回遊ばせて勝率を出す
3. チップを賭けられるようにし、勝てば2倍、引き分けなら賭けたぶんが戻るようにする

直す箇所が少なく、1か所に集まっているほど、変更に強い設計です。BlackjackGame を直す必要があったかどうかにも注目しましょう。

演習 ・ 付録課題

手札の点数をテストする

Hand を切り出したので、ゲームを動かさなくても点数の数え方を確かめられます。src/test/java/blackjack/HandTest.java に JUnit 5 のテストを書き、./mvnw -q test で実行しましょう。

手札	期待する点数
K, 7	17
A, K	21
A, A	12
A, A, 9	21
A, 6, K	17
K, Q, 2	22（バースト）

どのケースが A の数え方の法則を確かめているかを、テストの名前で言い表してみましょう。